

These functions are often implemented using `compare_and_swap()` operations. As an example, the following increments the atomic integer sequence:

```
increment(&sequence);
```

where the `increment()` function is implemented using the CAS instruction:

```
void increment(atomic_int *v)
{
    int temp;

    do {
        temp = *v;
    }
    while (temp != compare_and_swap(v, temp, temp+1));
}
```

It is important to note that although atomic variables provide atomic updates, they do not entirely solve race conditions in all circumstances. For example, in the bounded-buffer problem described in Section 6.1, we could use an atomic integer for count. This would ensure that the updates to count were atomic. However, the producer and consumer processes also have `while` loops whose condition depends on the value of count. Consider a situation in which the buffer is currently empty and two consumers are looping while waiting for `count > 0`. If a producer entered one item in the buffer, both consumers could exit their `while` loops (as count would no longer be equal to 0) and proceed to consume, even though the value of count was only set to 1.

Atomic variables are commonly used in operating systems as well as concurrent applications, although their use is often limited to single updates of shared data such as counters and sequence generators. In the following sections, we explore more robust tools that address race conditions in more generalized situations.

6.5 Mutex Locks

The **hardware-based solutions** to the **critical-section problem** presented in Section 6.4 are complicated as well as **generally inaccessible to application programmers**. Instead, operating-system designers build higher-level software tools to solve the critical-section problem. The simplest of these **tools** is the **mutex lock**. (In fact, **the term *mutex* is short for *mutual exclusion***.) **We use the mutex lock to protect critical sections and thus prevent race conditions**. That is, a process **must acquire the lock** before **entering** a critical section; it releases the lock when it exits the critical section. The `acquire()` function acquires the lock, and the `release()` function releases the lock, as illustrated in Figure 6.10.

A mutex lock has a boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to `acquire()` succeeds, and the lock is then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released.

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

Figure 6.10 Solution to the critical-section problem using mutex locks.

The definition of `acquire()` is as follows:

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;  
}
```

The definition of `release()` is as follows:

```
release() {  
    available = true;  
}
```

Calls to `either acquire() or release()` must be performed atomically. Thus, `mutex locks can be implemented using the CAS operation` described in Section 6.4, and we leave the description of this technique as an exercise.

LOCK CONTENTION

Locks are either contended or uncontended. A lock is considered **contended** if a thread blocks while trying to acquire the lock. If a lock is available when a thread attempts to acquire it, the lock is considered **uncontended**. Contended locks can experience either **high contention** (a relatively large number of threads attempting to acquire the lock) or **low contention** (a relatively small number of threads attempting to acquire the lock.) Unsurprisingly, highly contended locks tend to decrease overall performance of concurrent applications.

WHAT IS MEANT BY “SHORT DURATION”?

Spinlocks are often identified as the locking mechanism of choice on multiprocessor systems when the lock is to be held for a short duration. But what exactly constitutes a *short duration*? Given that waiting on a lock requires two context switches—a context switch to move the thread to the waiting state and a second context switch to restore the waiting thread once the lock becomes available—the general rule is to use a spinlock if the lock will be held for a duration of less than two context switches.

The main disadvantage of the implementation given here is that it requires busy waiting. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to `acquire()`. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU core is shared among many processes. Busy waiting also wastes CPU cycles that some other process might be able to use productively. (In Section 6.6, we examine a strategy that avoids busy waiting by temporarily putting the waiting process to sleep and then awakening it once the lock becomes available.)

The type of mutex lock we have been describing is also called a spinlock because the process “spins” while waiting for the lock to become available. (We see the same issue with the code examples illustrating the `compare_and_swap()` instruction.) Spinlocks do have an advantage, however, in that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. In certain circumstances on multicore systems, spinlocks are in fact the preferable choice for locking. If a lock is to be held for a short duration, one thread can “spin” on one processing core while another thread performs its critical section on another core. On modern multicore computing systems, spinlocks are widely used in many operating systems.

In Chapter 7 we examine how mutex locks can be used to solve classical synchronization problems. We also discuss how mutex locks and spinlocks are used in several operating systems, as well as in Pthreads.

6.6 Semaphores

Mutex locks, as we mentioned earlier, are generally considered the simplest of synchronization tools. In this section, we examine a more robust tool that can behave similarly to a mutex lock but can also provide more sophisticated ways for processes to synchronize their activities.

A semaphore `S` is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`. Semaphores were introduced by the Dutch computer scientist Edsger Dijkstra, and such, the `wait()` operation was originally termed `P` (from the Dutch